Cyber Shujaa

Security Analyst

ASSIGNMENT REPORT

HTB - Vulnerability Assessment

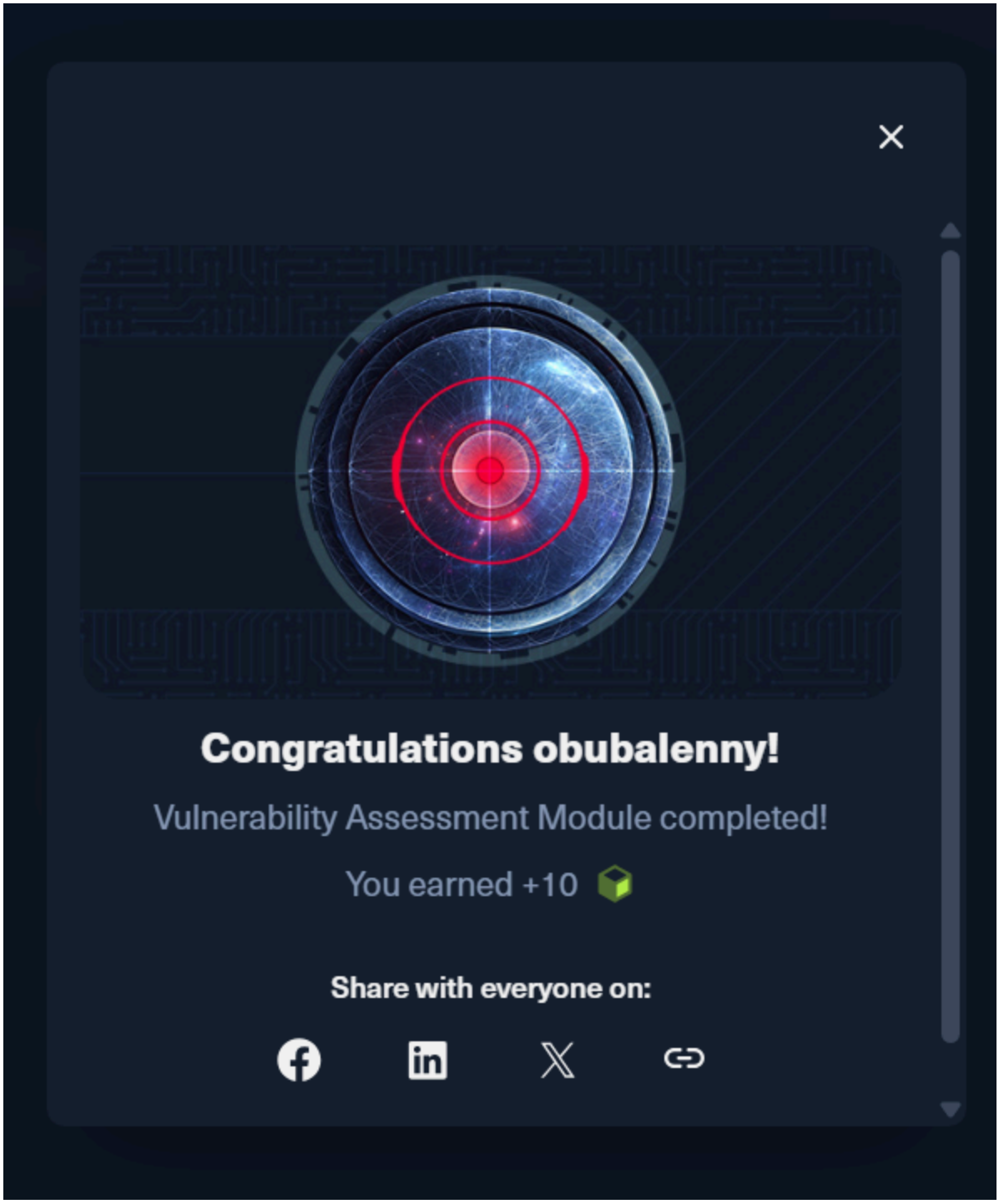
Full name	*Lenson Obuba*
Admission number	*cs-sa13-26017*
Email	*obubalenny@gmail.com*
Week and assignment	*Week 5, Assignment 2: Vulnerability Assessment*
Platform	*Hack The Box*
Date submitted	*11/10/2026*

1. Module completion

Shareable link

https://academy.hackthebox.com/achievement/2865856/108

Completion screenshot


Congratulations obubalenny!
Vulnerability Assessment Module completed!
You earned +10
Share with everyone on:

2. Introduction

This module covered the essential foundations of Vulnerability Assessments, focusing on how to conduct assessments, interpret findings, differentiate them from penetration tests, and deliver clear reporting. For a security analyst, mastering this workflow is critical because proactive vulnerability management enables organizations to identify and remediate security flaws before they can be actively exploited. Through this course, I expected to gain a structured, end-to-end framework for the assessment lifecycle, allowing me to translate technical scan data into actionable security insights for stakeholders.

3. The work

3.1 Security Assessments

What this section covered

This section introduces the foundational concepts of security assessments, specifically distinguishing between checklist-driven vulnerability assessments and simulated-attack penetration testing. The core takeaway is that an organization's security maturity level dictates which assessment is appropriate; lower-maturity entities should prioritize baseline vulnerability management before progressing to active exploit simulations to avoid overwhelming remediation teams. Ultimately, understanding these distinct methodologies ensures that security testing directly aligns with an organization's specific risk tolerance, compliance requirements, and operational capabilities.

3.2 Vulnerability Assessment

What this section covered

This section defines the core methodology of a vulnerability assessment, emphasizing its non-invasive nature and primary goal of identifying, categorizing, and prioritizing flaws without active exploitation. It breaks down critical risk calculation principles by explaining how vulnerabilities, threats, and exploits interact, using a qualitative likelihood-impact matrix to determine remediation priority. Finally, it highlights comprehensive asset management and

continuous inventory across on-premises, cloud, and SaaS environments as the mandatory starting point for any defensive security strategy.

3.3 Assessment Standards

What this section covered

This section outlines the regulatory frameworks and industry benchmarks governing both vulnerability assessments and penetration testing. It highlights key compliance standards, such as PCI DSS for cardholder environments, HIPAA for patient data, FISMA for government operations, and ISO 27001 for global security management, emphasizing that while compliance is mandatory for accreditation, a vulnerability management program must ultimately be driven by an organization's unique risk appetite. Additionally, it details standard penetration testing execution frameworks like PTES, OSSTMM, NIST, and OWASP, reinforcing the necessity of strict scopes, legal contracts, and non-destructive testing.

3.4 Common Vulnerability Scoring System (CVSS)

What this section covered

This section breaks down the industry-standard methodologies for calculating and calculating security threat severity, focusing heavily on CVSS v3.1 and Microsoft's DREAD framework. It details the three primary CVSS metric groups: Base Metrics (technical exploitability and CIA Triad impact), Temporal Metrics (exploit availability, remediation status, and reporting confidence), and Environmental Metrics (customizations based on an organization's specific asset importance). The primary takeaway is that while automated scanners assign these scores natively, analysts must understand how they are derived to manually calculate or justify adjustments based on a company's specific risk posture.

3.5 Common Vulnerabilities and Exposures (CVE)

What this section covered

This section introduces standardized methods for identifying, encoding, and cataloging security vulnerabilities using OVAL and CVE. It explains how OVAL uses XML definitions across four classes (Vulnerability, Compliance, Inventory, and Patch) to evaluate a system's state non-invasively, facilitating automation within frameworks like NIST's SCAP. It also outlines the lifecycle of a CVE identifier, highlighting the process of responsible disclosure where researchers work alongside vendors to ensure a patch is available before public disclosure, thereby preventing malicious exploitation of zero-day vulnerabilities.

3.6 Vulnerability Scanning Overview

What this section covered

This section explains how automated vulnerability scanning systematically identifies known network and application flaws without exploiting them. The takeaway is that scanners rely on both version checks and dynamic payloads to detect vulnerabilities, but they strictly require manual human verification to eliminate false positives. Ultimately, continuous authenticated and unauthenticated scanning is essential to keep patch management programs updated against evolving threats.

3.7 Getting Started with Nessus

What this section covered

This section provides a step-by-step guide to downloading, installing, and initializing Nessus Essentials via a .deb package on a Linux system. The key takeaway is understanding how to register for a free activation code, launch the background nessusd.service, and navigate the initial web GUI setup to compile critical plugins and configure advanced scanner options. Crucially, the module highlights that a pre-configured instance is available later in the skills assessment, allowing students to bypass local deployment and jump straight into tactical operations using provided credentials.

3.8 Nessus Scan

What this section covered

This section outlines how to configure a new Nessus scanner job by selecting from Discovery, Vulnerability, or Compliance templates, specifically highlighting the Basic Network Scan against target hosts. It details important tactical choices under the Discovery, Assessment, and Advanced tabs, such as disabling scans on fragile devices like printers to prevent network disruptions, and maintaining "safe checks" by default to run non-destructive audits. The takeaway is that while these scans can take hours, an analyst can actively triage findings in real time, customize port ranges, enable web crawler parameters, or even toggle user enumeration methods like RID brute-forcing.

3.9 Advanced Settings

What this section covered

This section highlights how to optimize vulnerability scans by tailoring Scan Policies, filtering out intentional false positives via custom Plugin Rules, and enabling deeply invasive auditing using

Credentialed Scanning. The takeaway is that building re-usable, user-defined templates and utilizing specific asset credentials (such as SSH, NTLM hashes, or database logins) shifts scanner behavior from surface-level banner grabbing to comprehensive, high-privilege configuration auditing. Furthermore, managing the reporting output by deliberately hiding non-exploitable findings or expected behaviors (like Microsoft DirectAccess cipher suites) ensures the final deliverable remains clear, concise, and focused on severe, actionable threats.

3.10 Working with Nessus Scan Output

What this section covered

This section covers managing raw and styled outputs from Nessus scans, highlighting the process of exporting data into structured formats like .pdf, .html, .csv, .nessus (XML), or .db files. The takeaway is that while these reports group critical findings by host and severity, they serve exclusively as supplementary appendices rather than final client deliverables. By utilizing automation scripts via the Nessus REST API or pipelines like EyeWitness, security analysts can systematically extract raw scan data, feeding it directly into triage tables, analytics platforms, or visual confirmation tools to streamline remediation.

3.11 Scanning Issues

What this section covered

This section highlights the potential operational risks of automated vulnerability scanning, such as network congestion, system instabilities, and firewall-induced false positives. The key takeaway is that analysts must actively mitigate these risks by communicating with stakeholders to schedule tests outside peak hours, skipping unsafe Denial of Service (DoS) checks, and enforcing strict rate-limiting on concurrent checks. Furthermore, as monitored using utilities like vnstat, the significant packet volume generated during active scanning emphasizes the absolute necessity of optimizing configurations to safeguard fragile or legacy network components.

3.12 Nessus Skills Assessment

What this section covered

This section presents a hands-on internal vulnerability assessment scenario for a corporate client's development Windows Server (172.16.16.100) to identify high-impact risks that can help justify future security budget increases. The assignment requires deploying a high-privilege, credentialed Basic Network Scan modified to audit all ports using the provided pre-configured Nessus laboratory interface. Ultimately, the exercise focuses on interpreting real-world scan

outputs to solve specific verification questions regarding exposed SMB shares, running VNC ports, and high-severity software vulnerabilities.

Question 1

What is the name of one of the accessible SMB shares from the authenticated Windows scan?

Answer

wsus

How you got there

To determine the accessible SMB share name, I logged into the pre-configured Nessus Essentials web application interface at port 8834 using the student credentials provided. Once inside, I navigated to the My Scans folder and opened the pre-populated authenticated scan profile titled Windows_basic_authed. From there, I drilled down into the specific target host entry (172.16.16.100) and opened its comprehensive Vulnerabilities analysis tab.

Within the vulnerability database results, I utilized the top inline search filter to isolate data related to "SMB Shares" and opened the specific plugin entry labeled "Microsoft Windows SMB Shares Access". By evaluating the detailed plugin output data block generated from the scan's administrative context, I verified the exact name of the exposed, read-accessible directory structure on the target server.

Evidence

The screenshot displays the specific Microsoft Windows SMB Shares Access plugin inspection window inside the Windows_basic_authed scan results.



It verifies that the wsus share configuration allows administrative read permissions, confirming exposure to authenticated network entities.

Question 2

What was the target for the authenticated scan?

Answer

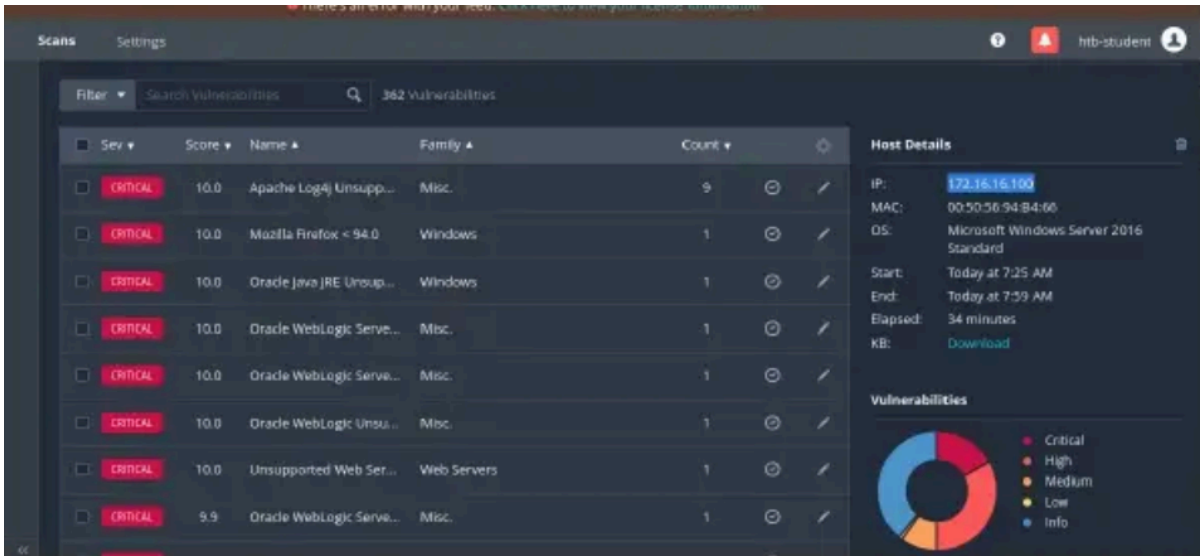
172.16.16.100

How you got there

To identify the network target assigned to the high-privilege configuration audit, I navigated to the Nessus dashboard and inspected the global execution parameters of the pre-run report. Within the overview properties of the Windows_basic_authed scan entry, the server configuration explicitly lists the singular destination IP address assigned to the scope. This confirms the infrastructure endpoint subjected to the credentialed testing run.

Evidence

The screenshot displays the main overview configuration page of the completed Windows_basic_authed scan dashboard panel.



It proves that the targeted evaluation parameters were explicitly directed to execute against the host IP address 172.16.16.100.

Question 3

What is the plugin ID of the highest criticality vulnerability for the Windows authenticated scan?

Answer

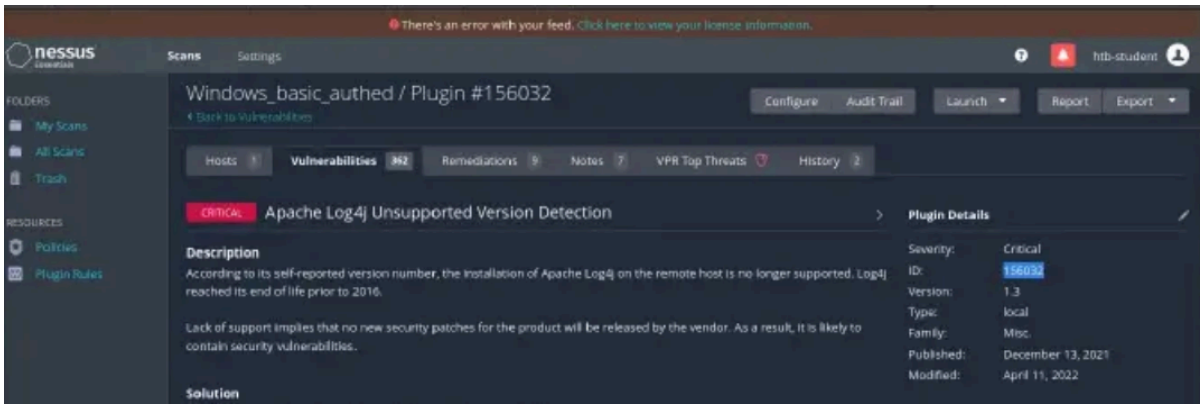
156032

How you got there

To determine the most severe vulnerability discovered on the system, I opened the completed Windows_basic_authed scan report and accessed the main findings dashboard. I filtered the results by severity rating to display the most severe issues at the top of the log, isolating the vulnerabilities categorized within the "Critical" band. From this prioritized list, I inspected the topmost entry, which corresponds to a critical remote code execution vulnerability and extracted its unique Nessus registration metric from the plugin information block.

Evidence

The screenshot displays the vulnerability list inside the Windows_basic_authed scan dashboard, sorted by severity with the highest criticality finding open.



It proves that the vulnerability tracker identified plugin ID 156032 as the highest priority security flaw affecting the Windows server environment.

Question 4

What is the name of the vulnerability with plugin ID 26925 from the Windows authenticated scan? (Case sensitive)

Answer

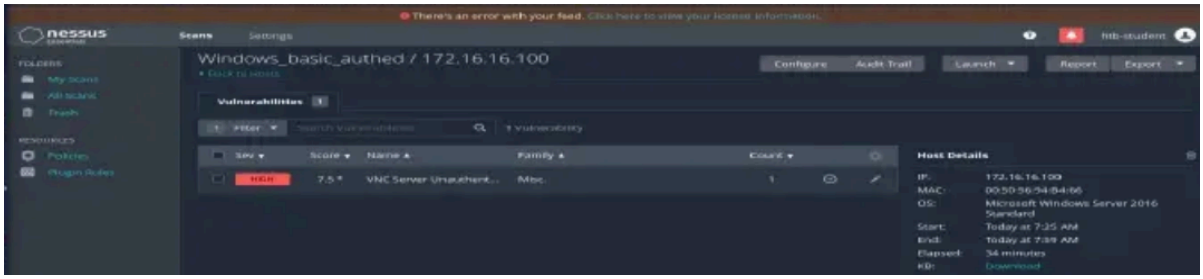
VNC Server Unauthenticated Access

How you got there

To identify the specific vulnerability corresponding to the provided tracker ID, I logged back into the active scanner session and pulled up the vulnerability inventory for the target host. I configured a custom search filter targeting the exact metadata parameter where the Plugin ID equals 26925. Once the log filtered out all other items, I selected the remaining high-severity entry to expand its detailed identification sheet, verifying the exact case-sensitive title assigned to the exposure by the tracking database.

Evidence

The screenshot displays the vulnerability profile details for Nessus plugin ID 26925 within the active laboratory interface.



It proves that the identification signature corresponds precisely to the case-sensitive string VNC Server Unauthenticated Access.

Question 5

What port is the VNC server running on in the authenticated Windows scan?

Answer

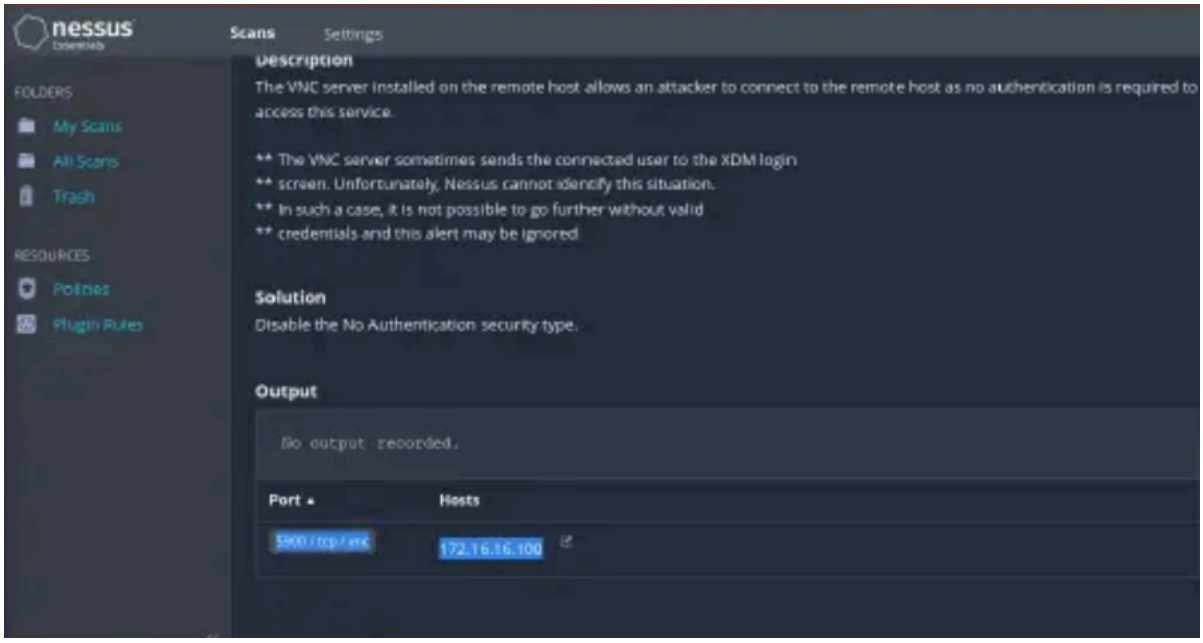
5900

How you got there

To determine the communication port utilized by the unauthenticated remote desktop service, I audited the active service signatures logged under the target host's open vulnerability report profile. I targeted the high-priority alert block associated with Plugin ID 26925 (VNC Server Unauthenticated Access) and parsed its metadata header properties. This specific configuration log verified that the insecure application layer endpoint binds directly to the default IANA network port designated for virtual network computing traffic.

Evidence

The screenshot displays the vulnerability information and execution metadata for the VNC exploit check inside the target's Nessus dashboard panel.



It proves that the scanner successfully detected the unauthenticated VNC instance listening directly over network TCP port 5900.

3.13 Getting Started with OpenVAS

What this section covered

This section introduces the installation and deployment process for OpenVAS via the Greenbone Vulnerability Manager (GVM) framework on a Linux system. The key takeaway is learning how to package-install the platform, initialize its multi-tier database structure using gvm-setup, and boot the dashboard services using the gvm-start command. Additionally, the text notes that an automated laboratory deployment is provided for the final exercises, letting security analysts connect instantly to port 8080 with standard training profile credentials.

3.14 OpenVAS Scan

What this section covered

This section explains how to manage scanning metrics, targets, and execution properties inside the Greenbone Security Assistant interface for OpenVAS. The core takeaway is learning how to isolate distinct scanning profiles (such as Base, Discovery, and Full and Fast) and assign administrative target credentials to exhaustively extract host vulnerability data. Ultimately, it emphasizes that by leveraging Network Vulnerability Test (NVT) families, analysts can strategically audit services while safely avoiding dangerous templates that could trigger system disruptions.

3.15 Exporting The Results

What this section covered

This section explains how to access, review, and export comprehensive scan reports within the Greenbone Security Assistant interface across multiple file formats like XML, CSV, and PDF. The takeaway is that raw XML exports can be efficiently parsed into scannable spreadsheets using external automation utilities like openvasreporting. This structured data transformation streamlines target triage by organizing critical network exposures, open ports, and CVSS severity metrics into clear charts that are easily digested by remediation teams.

3.16 OpenVAS Skills Assessment

What this section covered

This practical challenge assigns an internal Linux server assessment (172.16.16.160) for the client using a credentialed, high-privilege Full and Fast OpenVAS scanning workflow. The objective is learning how to log into the web management portal on port 8080, assign the correct root authentication profiles, and analyze the resulting vulnerability logs. Ultimately, the exercise reinforces the importance of interpreting these specific automated scanner findings to deliver strategic risk data that helps justify corporate security budget expansions.

Question 1

What type of operating system is the Linux host running? (one word)

Answer

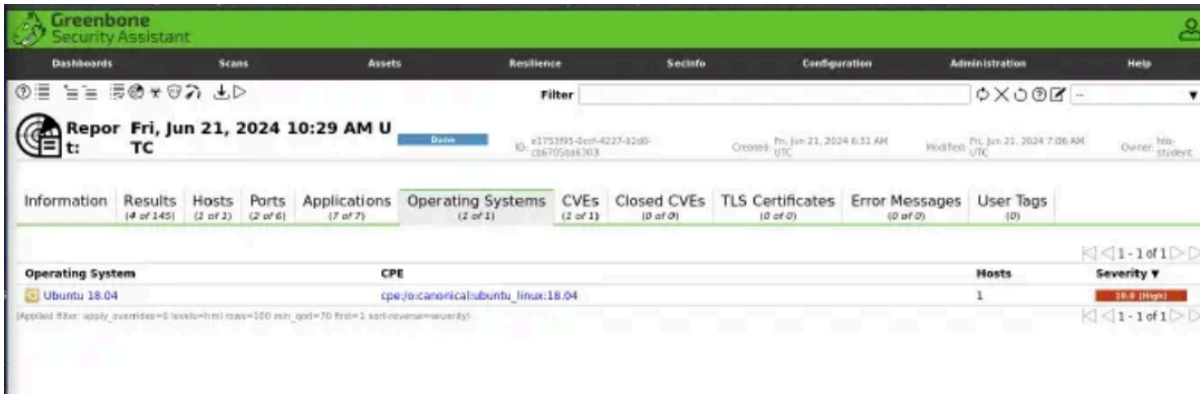
Ubuntu

How you got there

To determine the operating system layout of the remote server, I accessed the Greenbone Security Assistant interface on port 8080 via my Hack The Box OpenVPN Tunnel. From the left-hand navigation menu, I loaded the Scans page and checked the pre-populated report properties matching the active target IP (172.16.16.160). I filtered the host attributes by navigating to the Hosts configuration page, where the fingerprinting signatures explicitly identify the specific Linux distribution operating on the target architecture.

Evidence

The screenshot displays the target metrics and host detection data grid inside the active OpenVAS dashboard workspace.



It proves that the scanner's automated operating system discovery module verified the target asset signature as Ubuntu.

Question 2

What type of FTP vulnerability is on the Linux host? (Case Sensitive, four words)

Answer

Anonymous FTP Login Reporting

How you got there

To discover the exact classification of the FTP exposure, I loaded the completed report matching the active target IP within the OpenVAS dashboard. I navigated to the Results findings tab to analyze the full list of detected service vulnerabilities on the system. By sorting the detected anomalies and reviewing the precise application-layer alerts, I extracted the official four-word diagnostic title assigned to the insecure file transfer configuration.

Evidence

The screenshot displays the OpenVAS vulnerability log results grid with the active high-severity network findings filtered for the targeted host.



It proves that the scanner successfully detected an open file transfer portal, explicitly registering the classification title as Anonymous FTP Login Reporting.

Question 3

What is the IP of the Linux host targeted for the scan?

Answer

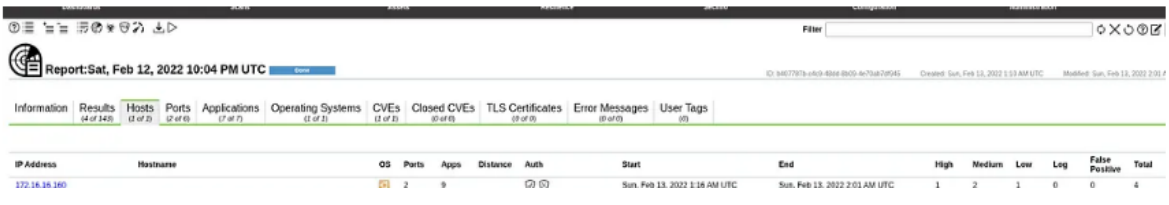
172.16.16.160

How you got there

To verify the explicit destination address designated for this security audit, I navigated to the Tasks overview inside the Greenbone Security Assistant dashboard page. I located the active scanner target configuration profile assigned to the evaluation execution window. Under the target parameters property card, the asset registration configurations explicitly validate the static internal network routing string assigned to the targeted remote Linux host architecture.

Evidence

The screenshot displays the OpenVAS task configurations and network target settings log dashboard.



It proves that the vulnerability evaluation run was explicitly directed to scan the target IP address 172.16.16.160.

Question 4

What vulnerability is associated with the HTTP server? (Case-sensitive)

Answer

Cleartext Transmission of Sensitive Information via HTTP

How you got there

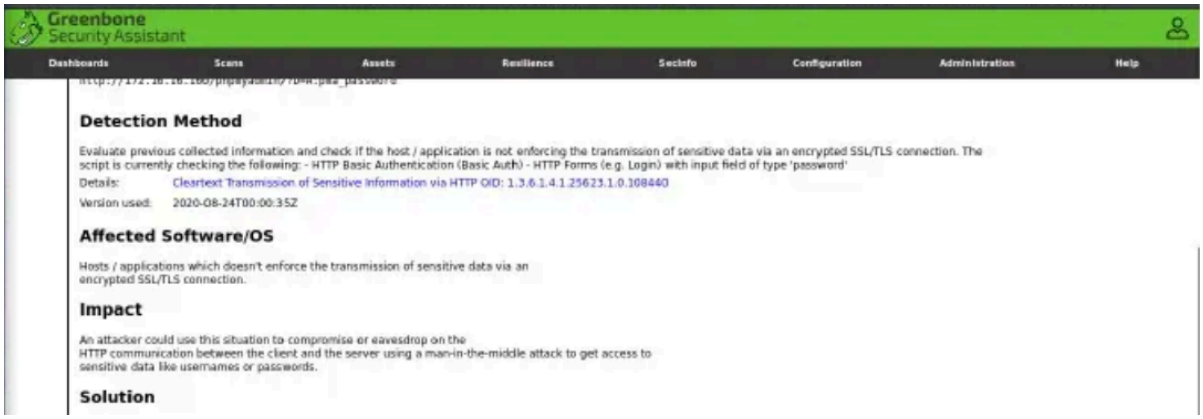
To locate the exact vulnerability affecting the web server instance, I examined the complete OpenVAS vulnerability list under the Results dashboard window. I filtered the scanning entries down to inspect network services bound explicitly to port 80 (HTTP). This isolated the specific

high/medium security alert rule containing the precise case-sensitive naming convention applied to unencrypted plaintext transport channels on the host.

Evidence

> *One sentence saying what the screenshot shows, then the screenshot, then one sentence on what it proves.*

The screenshot displays the detailed inspection page inside the Greenbone Security Assistant interface, detailing the Detection Method, Impact, and technical details for Cleartext Transmission of Sensitive Information via HTTP (OID



It proves that the explicit case-sensitive tracking alert label registered by the scanner's diagnostic check is Cleartext Transmission of Sensitive Information via HTTP

3.17 Reporting

What this section covered

This section highlights the critical role of strong communication and soft skills in transforming raw, automated scan data into a polished, client-ready Vulnerability Assessment Report [INDEX]. It details a comprehensive framework structured around an Executive Summary, methodology overview, defined scope, and validated findings grouped by severity or category [INDEX]. The takeaway is that a high-quality report must remain clear and accessible to both technical and non-technical stakeholders, ensuring every documented vulnerability features precise descriptions, CVSS scores, remediation guidance, and concrete proofs of concept [INDEX].

4. Challenges and how you resolved them

The primary operational challenge encountered during this assessment involved establishing a stable network connection to the remote laboratory environments. Initially, the browser-based Pwnbox environment was deployed, but restrictive resolution scaling and window-scrolling limitations made navigating the Nessus configuration pages highly inefficient. To resolve this, the workflow was shifted to a local host operating system utilizing an OpenVPN tunnel.

*During the initial terminal configuration on the local machine, the connection string failed due to syntax and directory path mismatches (Error opening configuration file). This hurdle was resolved by manually navigating into the local downloads directory, verifying the precise, customized filename using the ls *.ovpn listing utility, and launching the process with explicit administrative privileges via sudo openvpn.*

5. Conclusion

Completing this module has significantly shifted my perspective on automated scanning, moving me from blindly relying on tool output to critically evaluating and prioritizing risks myself. I can now confidently configure complex discovery profiles, manage high-privilege credentialed audits in both Nessus and OpenVAS, and manipulate plugin behaviors to completely filter out distracting false positives. What I learned was how much network strain and business disruption an improperly tuned scan can cause, emphasizing why a defensive analyst must carefully balance rate limits and safe checks rather than using default settings. Next time, I plan to proactively coordinate with host administrators before scanning to optimize target scopes, which mirrors a security analyst's real-world duty to turn raw vulnerability logs into safe, clear, and actionable remediation strategies for an enterprise.